



FAST National University of Computer and Emerging Sciences

Karachi Campus | Department of Computer Science

FINAL PROJECT REPORT

Google Classroom Clone

A Full-Stack C++ OOP Console Application Simulating a Learning Management System with Role-Based Access, Stream, Classwork, Online Class, and Grade Tracking

Course:	Object Oriented Programming — Lab Project
Instructor:	Sir Muhammad Khalid Khan
Section:	BAI-2B
Semester:	Spring 2026
Submission Date:	April 2026

Group Members

#	Name	Roll Number	Role
1	Taimoor (Lead)	25K-0096	Architecture & Full Implementation
2	Zain Noman	25K-0146	Testing & Documentation
3	Abdul Farid	25K-0029	Demo Data & Code Review

FAST-NUCES Karachi | Spring 2026 | Final Project Report

Table of Contents

1.Executive Summary3

2.Introduction3

- 2.1 Background & Motivation3
- 2.2 Problem Statement4
- 2.3 Objectives4
- 2.4 Scope4

3.System Analysis5

- 3.1 Functional Requirements5
- 3.2 Non-Functional Requirements5
- 3.3 Use Cases5

4.System Design6

- 4.1 Class Hierarchy6
- 4.2 UML Class Relationships6
- 4.3 Array Capacity Constants6

5.Implementation7

- 5.1 Technology Stack7
- 5.2 OOP Concepts Implemented7
- 5.3 Module-wise Implementation8
- 5.4 Key Code Snippets8

6.Testing & Results9

- 6.1 Compilation Test9
- 6.2 Functional Testing9
- 6.3 OOP Concept Verification9

7.Challenges & Solutions10

8.Division of Work11

9.Conclusion & Future Work11

- 9.1 Conclusion11
- 9.2 Future Work12

10.References12

1. Executive Summary

This report presents the design, development, and evaluation of the Google Classroom Clone, a multi-class C++ OOP console application developed as the final project for the Object Oriented Programming Lab course at FAST National University of Computer and Emerging Sciences, Karachi, under the supervision of Sir Muhammad Khalid Khan.

The system models a complete Learning Management System (LMS) where Teachers and Students interact through three classroom tabs Stream, Classwork, and Online Class using a fully object-oriented architecture. It demonstrates all core OOP concepts including inheritance, polymorphism, encapsulation, composition, and dynamic dispatch.

The final system includes modules for posting announcements, assignment creation and submission, quiz management, recording uploads, meeting scheduling, grade tracking, and a platform-level login system all implemented without the STL vector container, using fixed-size arrays exclusively.

2. Introduction

2.1 Background & Motivation

Modern educational institutions use Learning Management Systems (LMS) like Google Classroom to manage academic workflow announcements, assignments, quizzes, recorded lectures, and live meetings from a single platform. Building such a system in C++ provides an ideal vehicle for demonstrating object-oriented design: each real-world academic entity maps naturally to a class, and the relationships between them illustrate inheritance, composition, and polymorphism in a meaningful context.

2.2 Problem Statement

No existing console-based C++ project in the course demonstrated all of the following simultaneously: multi-level inheritance, runtime polymorphism, composition, encapsulation, dynamic dispatch, and a complete interactive UI all without relying on STL vectors. Key challenges included:

- Designing a class hierarchy that is genuinely extensible (new teacher types or classroom modules can be added with minimal changes).
- Storing all dynamic data in fixed-size arrays with manual count tracking instead of STL containers.
- Implementing a platform-level login that returns a `User*` base pointer and correctly routes to the right menu via `dynamic_cast`.
- Providing separate, role-appropriate menu systems for Teachers and Students while sharing the same Classroom objects.

2.3 Objectives

1. Design and implement a full class hierarchy: User base class with Student and Teacher derived classes, and four teacher specializations.
2. Implement all core OOP concepts: inheritance, polymorphism, encapsulation, composition, and dynamic dispatch.
3. Replace all STL vector usage with fixed-size arrays and manual count variables.
4. Provide interactive console menus for Teacher and Student roles navigating Stream, Classwork, and Online Class tabs.
5. Demonstrate runtime polymorphism through virtual `conductSpecialActivity()` and `getRole()` methods.
6. Deliver zero-warning compilable code under C++17 with g++.

2.4 Scope

In Scope: Full class hierarchy, all three classroom tabs, grade tracking, platform login, four derived teacher types, menu-driven UI, seed demo data, and all OOP concept demonstrations.

Out of Scope: File persistence (data resets on exit), graphical UI, network communication, and STL containers beyond string.

3. System Analysis

3.1 Functional Requirements

ID	Requirement	Priority
FR01	Users must log in with ID and password; system routes to Teacher or Student menu based on role.	High
FR02	Teachers can post announcements, view streams, and read student posts.	High
FR03	Teachers can create assignments (title, description, due date, marks) and quizzes.	High
FR04	Students can submit assignments and view all classwork.	High
FR05	Teachers can view submissions and assign marks; system records graded status.	High
FR06	Teachers can upload recordings (title, URL, duration) and schedule meetings.	Medium
FR07	Teachers can start and end meetings; status shown as LIVE or Upcoming.	Medium
FR08	Students can view their grades across all enrolled classes.	Medium
FR09	Dashboard shows live meeting count, post count, and assignment/quiz summary per classroom.	Low
FR10	Derived teacher classes (Math, Science, English, CS) demonstrate unique special activities.	Low

3.2 Non-Functional Requirements

- Correctness: All OOP concepts must be demonstrably present and functional.
- Compilability: Code must compile with zero errors and zero warnings under g++ -std=c++17 -Wall -Wextra.
- No STL Vectors: All dynamic collections must use fixed-size arrays with MAX_* constants.
- Usability: Menus must be hierarchical and navigable with numeric choices.
- Maintainability: New teacher types or classroom modules must be addable with minimal changes.
- Memory Safety: All heap-allocated objects must be deleted in Platform destructor.

3.3 Use Cases

The two main actors interact with the system as follows:

- Teacher: Logs in; navigates to a classroom; posts announcements; creates assignments and quizzes; uploads recordings; schedules and manages meetings; views and grades submissions; views their profile and conducts a special activity unique to their subject.
- Student: Logs in; views any classroom dashboard; reads the stream and posts questions; views and submits assignments; views quizzes; watches recordings and joins meetings; views their own grades across all enrolled classes.

4. System Design

4.1 Class Hierarchy

The application follows a layered OOP architecture:

- **Data Classes:** Post, Assignment, Quiz, Recording, Meeting, Submission, GradeTracker — each encapsulates a specific academic entity.
- **Module Classes:** Stream, Classwork, OnlineClass composite classes that own arrays of the data classes and expose operations on them.
- **User Hierarchy:** User (base) → Student, Teacher → MathTeacher, ScienceTeacher, EnglishTeacher, CSTeacher.
- **Orchestrator:** Classroom — owns one of each module class and a Teacher pointer; provides the unified interface used by menus.
- **Platform:** Top-level singleton holding arrays of all students, teachers, and classrooms; provides login and lookup.

4.2 UML Class Relationships

From	Relationship	To
User	is-a base of	Student, Teacher
Teacher	is-a base of	Math Teacher, ScienceTeacher, EnglishTeacher, CSTeacher
Classroom	HAS-A	Stream, Classwork, OnlineClass (composition)
Classroom	HAS-A*	Teacher* (association via pointer)
Student	HAS-A[]	GradeTracker[MAX_ENROLLED] (composition)
Platform	HAS-A*[]	Student*[], Teacher*[], Classroom*[] (aggregation)
Stream	HAS-A[]	Post[MAX_POSTS]
Classwork	HAS-A[]	Assignment[], Quiz[], Submission[]
OnlineClass	HAS-A[]	Recording[], Meeting[]

4.3 Array Capacity Constants

Constant	Value	Controls
MAX_COMMENTS	20	Comments per Post
MAX_POSTS	50	Posts per Stream
MAX_ASSIGNMENTS	30	Assignments per Classroom
MAX_QUIZZES	20	Quizzes per Classroom
MAX_SUBMISSIONS	50	Submissions per Classroom
MAX_RECORDINGS	20	Recordings per Classroom
MAX_MEETINGS	20	Meetings per Classroom
MAX_ENROLLED	10	Classes a Student may join

MAX_STUDENTS_CLS	50	Students per Classroom
MAX_STUDENTS_PLT	100	Students on Platform

5. Implementation

5.1 Technology Stack

Layer	Technology	Purpose
Language	C++17	Core application and all OOP constructs
Paradigm	Object-Oriented Programming	Encapsulation, inheritance, polymorphism, composition
Collections	Fixed-size arrays (no vectors)	All dynamic data no STL vector used anywhere
I/O	cin / cout (console)	Interactive menu-driven text interface
Compiler	g++ / MinGW (C++17)	Build, validation, zero-warning target
Version Control	Git + GitHub	Collaboration, history, and code review

5.2 OOP Concepts Implemented

OOP Concept	Implementation Detail
Classes & Objects	15 classes total: Post, Assignment, Quiz, Recording, Meeting, Submission, GradeTracker, Stream, Classwork, OnlineClass, User, Student, Teacher, Classroom, Platform
Single Inheritance	Student : public User and Teacher : public User
Multi-level Inheritance	MathTeacher : public Teacher : public User three-level chain
Polymorphism	virtual conductSpecialActivity() in Teacher, overridden in MathTeacher, ScienceTeacher, EnglishTeacher, CSTeacher. virtual getRole() overridden in Student and Teacher.
Encapsulation	All data members are private or protected. Access only through public getters and controlled mutators.
Composition	Classroom owns Stream, Classwork, OnlineClass as value members. Students own GradeTracker[MAX_ENROLLED] as a value array.
Dynamic Dispatch	Platform::login() returns User*. dynamic_cast<Teacher*> and dynamic_cast<Student*> route to the correct menu.
Virtual Destructor	virtual ~User() {} ensures correct destruction when deleting derived objects through a base pointer in Platform destructor.
No STL Vectors	Every vector<T> is replaced with T arr[MAX_N] + int count. push_back becomes arr[count++]; size() becomes the count variable.

5.3 Module-wise Implementation

Module 1 — Authentication & Platform

Platform::login(id, pass) iterates the students array then the teachers array, comparing IDs and passwords. On success it returns a User* base pointer. main() calls dynamic_cast to route to teacherMenu() or studentMenu(). The Platform destructor deletes all heap-allocated classrooms, students, and teachers to prevent memory leaks.

Module 2 — User Hierarchy

The user holds name, id, email, password as protected strings and exposes getName(), getId(), getEmail(), checkPassword(), and virtual displayProfile() / getRole(). Students add enrolled class codes and GradeTracker arrays. The teacher adds subjectSpeciality and manages class codes. Four derived teacher classes each override conductSpecialActivity() with subject-specific behavior.

Module 3 — Stream

Stream owns Post[MAX_POSTS] with a postCount_ counter. Each Post holds author, content, timestamp, and a comments[MAX_COMMENTS] array. addPost() copies a freshly constructed Post into the array. display() iterates all posts and calls their display() methods.

Module 4 — Classwork

Classwork owns Assignment[MAX_ASSIGNMENTS], Quiz[MAX_QUIZZES], and Submission[MAX_SUBMISSIONS] arrays with separate count variables. gradeSubmission(idx, marks) sets the marks and flips isGraded.

Module 5 — Online Class

OnlineClass owns Recording[MAX_RECORDINGS] and Meeting[MAX_MEETINGS] arrays. Meeting stores title, link, scheduled time, and isLive flag with goLive() and endMeet() mutators. liveCount() iterates the meetings array to count currently active sessions.

Module 6 — Grade Tracker

GradeTracker owns a ScoreEntry[MAX_SCORES] array (replacing the original vector<pair<string,int>>). ScoreEntry is a plain struct holding title and marks. display() iterates all scores, prints them with set formatting, and computes the running total.

5.4 Key Code Snippets

Array-based collection pattern (replacing vector):

```
// Instead of: vector<Post> posts;
Post posts[MAX_POSTS];
int postCount_ = 0;

// Instead of: posts.push_back(Post(author, content));
if (postCount_ < MAX_POSTS)
    posts[postCount_++] = Post(author, content);
```

Virtual function override in derived teacher class:

```
class MathTeacher : public Teacher {
public:
    void conductSpecialActivity() override {
        cout << "[Math] Conducting a problem-solving quiz.\n";
    }
};
```

Dynamic dispatch after login:

```
User* user = platform.login(id, pass);
if (user->getRole() == "Teacher")
```

```
        teacherMenu(dynamic_cast<Teacher*>(user), platform);  
    else  
        studentMenu(dynamic_cast<Student*>(user), platform);
```

6. Testing & Results

6.1 Compilation Test

The final source file was compiled with the strictest warning flags:

```
g++ -std=c++17 -Wall -Wextra -o classroom classroom_no_vectors.cpp
```

Result: Zero errors, zero warnings. Exit code 0.

6.2 Functional Testing

Test Case	Input	Expected Output	Result
Valid Teacher Login	T001 / math123	Welcome, Mr. Khan! (Teacher)	Pass
Valid Student Login	S001 / pass1	Welcome, Ali Hassan! (Student)	Pass
Invalid Login	T001 / wrongpass	Invalid credentials.	Pass
Post Announcement	Teacher → Stream → 1	Post added to stream.	Pass
Add Assignment	Teacher → Classwork → 2	Assignment X added.	Pass
Submit Assignment	Student → Classwork → 2	Submission recorded.	Pass
Grade Submission	Teacher → Classwork → 6	Graded. isGraded = true.	Pass
View Grades	Student → My Grades	Grade report per class	Pass
Schedule Meeting	Teacher → Online → 4	Meeting scheduled.	Pass
Start / End Meeting	Teacher → Online → 5/6	LIVE NOW / Upcoming status	Pass
conductSpecialActivity()	MathTeacher Profile	[Math] problem-solving quiz.	Pass
Array bound check	Fill to MAX_POSTS	Stream is full. (no crash)	Pass

6.3 OOP Concept Verification

Concept	Verification Method	Result
Inheritance	Student* and Teacher* pointers stored as User*; getName() resolves correctly	Pass
Polymorphism	Loop over four derived teacher pointers calling conductSpecialActivity() each prints distinct output	Pass
Encapsulation	Direct access to name, id, password fails to compile only accessible via getters	Pass

Composition	Classroom object holds Stream/Classwork/OnlineClass by value destroyed with Classroom	Pass
Dynamic Dispatch	dynamic_cast<Teacher*>(user) returns non-null for T001; null for S001	Pass
No STL Vectors	grep 'vector' classroom_no_vectors.cpp returns zero matches	Pass

7. Challenges & Solutions

Challenge	Solution Applied
Removing all STL vectors	Defined MAX_* constants at file top; replaced every vector<T> with T arr[MAX_N] + int count; replaced push_back with bounds-checked arr[count++]; replaced range-for with index-for loops.
Replacing pair<string,int> in GradeTracker	Defined a plain ScoreEntry struct { string title; int marks; } cleaner and more readable than pair.
Default constructors required for arrays	Added default constructors to all data classes (Post, Assignment, Quiz, etc.) so they can be declared as fixed-size arrays.
Login routing with base pointer	Platform::login() returns User*. Role-based routing uses getRole() string comparison then dynamic_cast to the correct derived type.
Preventing memory leaks	Platform destructor iterates all three pointer arrays and deletes each object. Teacher pointers in the classroom are non-owning (Platform owns them).
cin/getline mixing	After every cin >> choice, cin.ignore() is called to consume the newline so subsequent getline() calls work correctly.

8. Division of Work

Member	Roll No.	Modules / Tasks	%
Taimoor (Lead)	25K-0096	Full class hierarchy design; all data, module, and orchestrator classes; Teacher/Student menus; polymorphism demonstration; vector-to-array conversion; compilation and testing	34%
Zain Noman	25K-0146	Seed demo data design; boundary and edge-case testing; viva question preparation; README documentation	33%
Abdul Farid	25K-0029	Final report writing; code commenting and cleanup; presentation slide deck; project submission packaging	33%

Note: All group members reviewed and tested each other's code. Every member is able to explain, modify, and debug any part of the entire project during the viva evaluation.

9. Conclusion & Future Work

9.1 Conclusion

The Google Classroom Clone successfully achieves all stated objectives. The project demonstrates practical application of all OOP concepts covered throughout the lab course from foundational class design to advanced runtime polymorphism and dynamic dispatch within a single, cohesive, compilable C++ application.

The deliberate constraint of eliminating STL vectors in favour of fixed-size arrays deepened our understanding of manual memory management, default constructors, and the trade-offs between static and dynamic data structures.

9.2 Future Work

- File persistence: Save and load all platform data to/from binary or CSV files so records survive between sessions.
- STL migration: Re-introduce `std::vector`, `std::map`, and `std::unordered_map` as an optional build target once the array-only constraint is lifted.
- GUI front-end: Connect the OOP backend to a Qt or SFML graphical interface.
- GPA computation: Auto-calculate GPA from all graded submissions and display a cumulative academic transcript.
- Role expansion: Admin role with full platform-level CRUD over users and classrooms.

10. References

7. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
8. Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). C++ Primer (5th ed.). Addison-Wesley.
9. cppreference.com. C++ Language Reference. <https://en.cppreference.com/>
10. ISO/IEC 14882:2017. Programming Language — C++17 Standard.
11. GeeksforGeeks. Object Oriented Programming in C++. <https://www.geeksforgeeks.org/>
12. Course Lab Materials — Sir Muhammad Khalid Khan, FAST-NUCES Karachi, Spring 2026.

Group Representative

Instructor

25K-0096 (Taimoor)

Sir Muhammad Khalid Khan

FAST National University of Computer and Emerging Sciences, Karachi | Spring 2026